



Personal Toolkit Applications

Practical Skills Module 6: Topic 2

[Visit our website](#)

Practical Skills Module 6: Topic 2

Topic Code	PM-06-PS02
Topic	Python exercises may be conducted at the end of the skills programme or may be integrated into learning at the end of the applicable module/section

Scope of Practical Skill

Given a range the learner must be able to:

- PA0201 Build a:
 - Alarm clock
 - Calculation of compounded interest
 - A GST (good service tax) calculator (GUI)
 - Language translator
 - Contact book
 - Create a scientific calculator in Python

Applied Knowledge

- AK0201 Concept, definition and functions of Python programming principles

Internal Assessment Criteria

- IAC0201 Expected results are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-06



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Personal Toolkit

You are tasked with developing a Personal Toolkit in Python. This toolkit consists of a set of utility applications designed to help individuals manage their daily life more efficiently. Each tool serves a specific purpose, ranging from time management and financial planning to communication and calculations. Your job is to create each of these tools separately, all organized in a single folder. Each step below reflects a key function in your toolkit:

Create a folder named **python_projects** and complete the following:

Step 1: Build an alarm clock

- Create a Python file called **alarm_clock.py**.
- Build a basic alarm clock that plays a sound at a specified time:
 - Asks the user to enter an alarm time in **HH:MM:SS** format.
 - Continuously checks the current time until it matches the alarm time.
 - Play a sound and print "Alarm ringing!" when the time matches.
 - You can use the `sleep()` function to pause the program to check the time every second.
- *Tip:* You can import the `datetime`, `time`, and `winsound` (for Windows) libraries to create this program. If you're using macOS or Linux, you may use an alternative like `playsound`.

Step 2: Build a compounded interest calculator

- Create a Python file called **compounded_interest.py**.
- Build a program to calculate compounded interest based on user input:

- Use the compound interest formula: $A = P * (1 + r/n)^{(n*t)}$
- Prompt the user to enter:
 - Principal amount (P)
 - Annual interest rate (as a percentage)(r)
 - Time in years (t)
 - Number of times compounded per year (n)
- Convert the annual interest rate from a percentage to a decimal before using it in calculations.
- Calculate and print out the total total amount (A) and the interest earned (A - P).

Step 3: Build a GST (good service tax) calculator (GUI)

- Create a Python file called **gst_calculator.py**.
- Use PyQt5 to calculate GST and create a GUI with:
 - An input for Original Price
 - An input for GST Rate (%)
 - A Calculate button
 - A label showing the GST Amount and Total Price
- Formula to calculate GST:
 - $\text{GST Amount} = (\text{Original Price} \times \text{GST Rate}) / 100$
 - $\text{Total Price} = \text{Original Price} + \text{GST Amount}$
- Handle invalid inputs with an error message.
- *Tip:* You can import the following classes from the PyQt5.QtWidgets module to build the interface: QApplication, QWidget, QLabel, QLineEdit, QPushButton, QVBoxLayout, QMessageBox

Step 4: Build a language translator

- Create a Python file called **language_translator.py**.
- Create a basic language translator using the **googletrans** library.
 - The program should:
 - Take user input text.
 - Take a target language code (e.g., 'fr' for French, 'es' for Spanish).
 - Translate the input into the specified language.

- Print the translated result.
- Run your program and show the output in the terminal.
- *Tip:* To avoid errors related to asynchronous calls, install a specific stable version of `googletrans` that uses synchronous calls: `'pip install googletrans==4.0.0-rc1'`.

Step 5: Build a contact book

- Create a Python file called **contact_book.py**.
- The program should store, retrieve, and display contact information.
- Use a dictionary to store contacts, where each key is a contact name and the value is another dictionary holding phone number and email.
- Implement a function called `add_contact` that:
 - Prompts the user to enter a name, phone number, and email.
 - Adds this information to the dictionary using the name as the key and a nested dictionary as the value.
 - Prints a confirmation message once the contact is added.
- Implement a function called `view_contacts()` that:
 - Loops through the dictionary.
 - Prints all stored contacts in a clear, readable format, including name, phone number, and email.
- Create a menu-driven interface in a loop that:
 - Displays the following options to the user:
 - Add Contact
 - View Contacts
 - Exit
 - Prompts the user to enter their choice.
 - Calls the appropriate function based on the user's choice.

Step 6: Build a scientific calculator

- Create a Python file called **scientific_calculator.py**.
- The console-based scientific calculator should perform basic and scientific operations such as:
 - Addition, Subtraction, Multiplication, Division
 - Square Root

- Power (Exponentiation)
- Trigonometric Functions: Sine, Cosine, Tangent (input in degrees)
- Use the `math` module to perform advanced calculations.
- Validate user input (e.g., prevent division by zero).
- Ensure that after each operation, the menu is shown again until the user chooses to exit.
- *Tip:* Use the `math` module for advanced calculations such as:
 - `math.sqrt(x)` for square roots
 - `math.pow(x, y)` for exponentiation
 - `math.sin()`, `math.cos()`, `math.tan()` for trigonometric functions
 - `math.radians(x)` to convert degrees to radians



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
